CSA4701-DEEP LEARNING

NAME : YUGITHA B

REG NO : 192424102

ASSIGNMENT 1 – CNN DESIGN & GRADIENT-BASED TRAINING

Assessment topic: Design and train a CNN for an image-classification problem. Compute output shapes and parameter counts for each layer, select suitable initialization and optimizer, apply regularization, and report learning curves and test results.

TASK 1 – Understand the Problem and State the Aim

Aim: To design, train and evaluate a Convolutional Neural Network (CNN) for image classification and to study its layer dimensions, trainable parameters, training method and ability to classify unseen images.

Problem statement: Given a labelled image dataset, the CNN must learn patterns from training images and predict the correct class for each new image. The final model must be evaluated on a separate test set.

TASK 2 – Select the Dataset and Prepare the Input

A suitable example is CIFAR-10, which contains 32 × 32 colour images belonging to 10 classes. If the faculty provides another dataset, use that dataset instead.

1. Load the images and their class labels.
2. Resize images only if the selected model requires a different fixed size.
3. Convert pixel values from 0–255 to values between 0 and 1 by dividing by 255.
4. Separate the data into training, validation and test sets. Use training data to learn, validation data to select settings, and test data only for final evaluation.
5. Apply augmentation only to training images when useful, for example small shifts or horizontal flips.

TASK 3 – Design the CNN Architecture

Layer	Configuration	Output shape for 32×32×3 input	Trainable parameters
Input	RGB image	32 × 32 × 3	0
Conv2D + ReLU	32 filters, 3×3, stride 1, same padding	32 × 32 × 32	(3×3×3 + 1)×32 = 896
Max pooling	2×2, stride 2	16 × 16 × 32	0
Conv2D + ReLU	64 filters, 3×3, stride 1, same padding	16 × 16 × 64	(3×3×32 + 1)×64 = 18,496
Max pooling	2×2, stride 2	8 × 8 × 64	0
Flatten	Convert feature map into a vector	4096	0
Dense + ReLU	128 units	128	(4096 + 1)×128 = 524,416
Dropout	Rate 0.5 during training	128	0
Dense + Softmax	10 classes	10	(128 + 1)×10 = 1,290

The total trainable parameter count for this example is 545,098. This count assumes 10 output classes and includes biases. If the dataset has a different number of classes, change the final dense layer and recalculate its parameters.

TASK 4 – Show the Mathematical Calculations

For a convolution or pooling layer, the output size is: floor((Input − Kernel + 2 × Padding) / Stride) + 1. With same padding and stride 1, a 3×3 convolution keeps the height and width unchanged. A 2×2 pooling layer with stride 2 halves both dimensions: 32 becomes 16, then 16 becomes 8.

For a convolution layer: Parameters = (kernel height × kernel width × input channels + 1) × number of filters. For a dense layer: Parameters = (number of input units + 1) × number of output units. The extra 1 is the bias for each output unit.

TASK 5 – Select Initialization, Optimizer and Loss Function

- Initialization: He initialization is a suitable starting choice for layers followed by ReLU.
- Optimizer: Adam with a starting learning rate such as 0.001. Tune the learning rate using validation performance.
- Loss: Sparse categorical cross-entropy when labels are integer class IDs; categorical cross-entropy when labels are one-hot encoded.
- Output activation: Softmax for a task in which each image belongs to one of several mutually exclusive classes.

TASK 6 – Describe the Training Algorithm

Pseudocode:

INPUT: training images X, labels y, number of classes C

Preprocess images and create training, validation and test sets

Build the CNN architecture

Initialize weights and compile with Adam and cross-entropy loss

FOR each epoch:

    Train on mini-batches from the training set

    Calculate validation loss and validation accuracy

    Save the model if validation performance improves

    Stop early if validation performance does not improve for the selected patience

Load the best saved model

Evaluate once on the test set

Report loss, accuracy, precision, recall, F1-score and confusion matrix

TASK 7 – Control Overfitting and Evaluate the Model

Overfitting occurs when training performance keeps improving but validation performance becomes worse. Dropout, data augmentation, early stopping and weight decay can help reduce it. Use only techniques that are appropriate for the observed learning curves.

- Accuracy = correct predictions / total predictions.

- Precision = TP / (TP + FP).
- Recall = TP / (TP + FN).
- F1-score = 2 × Precision × Recall / (Precision + Recall).
- Plot training and validation loss and accuracy against epochs. Use a confusion matrix to identify classes that are frequently confused.

TASK 8 – Record Experimental Results

Run a baseline model and at least one regularized or tuned model on the same dataset split. Do not fill the table using guessed values.

Experiment	Settings	Validation result	Test loss / accuracy / F1
Baseline CNN	Adam, learning rate 0.001	Enter measured values	Enter measured values
Regularized CNN	Dropout and/or augmentation	Enter measured values	Enter measured values
Tuned CNN	Best validation-selected settings	Enter measured values	Enter measured values

TASK 9 – Analyse Results and Defend the Final Design

Compare the models using validation results first, then report the test result of the selected model. Explain whether the learning curves show underfitting or overfitting, which classes are confused most often, and whether regularization improves generalization. The final recommendation must be supported by the measured results.

TASK 10 – Conclusion

A CNN learns visual features through convolution and pooling layers and uses dense layers to predict classes. Output-shape and parameter calculations help verify the architecture. Initialization, optimization and regularization support training, but the final conclusion about performance must be based on actual test results.

References

Cite the dataset used and the official documentation for the framework used, such as TensorFlow/Keras or PyTorch.